Full Stack Developer Intern Evaluation

Overview

You will build a small award judging app from scratch to a written spec, handle a requirement change partway through, review some AI-written code, and then defend your work live. AI tools are allowed. We grade what your app actually does under our tests, and what you can explain, not how much code you produce.

Item	Detail
Code	EVAL-INT-2026
Role	Full Stack Developer Intern
Stack	React, Node.js, PostgreSQL
Deadline	3.5 working days after you receive this test. Work around your studies; no need to do it in one sitting.
Expected effort	About 10 to 12 hours in total. More hours will not earn more marks.
Live defense	30 minutes, within 2 working days of submission
AI tools	Allowed: any IDE, LLM or coding assistant
Deliverables	Git bundle, README.md, DECISIONS.md, REVIEW.md, AI_LOG.md

Candidate name: _____

Ground rules

Use AI as much as you like. Every line you submit must be one you can explain, change and defend without it.

1. **Follow the API contract exactly.** Our test suite calls your API through the contract below, so a wrong path, field name or status code fails those tests however good the code is.
2. **Your own tests are not our tests.** We grade with a separate test suite, including concurrency, timezone, permission and pagination tests. Write your own tests too.
3. **Commit as you go.** Small commits with clear messages, on every day you work. We read the history. One huge commit of a finished app scores zero for process.

4. **If you think the spec is wrong,** build to it anyway and say why in DECISIONS.md.
5. **Keep an AI log.** In AI_LOG.md, record:
   - the main prompts you used, in one line each
   - every place the AI was wrong, incomplete or misleading, and how you found out
   - any AI suggestion you rejected, and why
6. **Work alone.** AI tools are fine; another person is not.

A polished AI_LOG.md that lists no AI mistakes is a red flag, not a strength. Catching the AI's errors earns marks.

Project setup

You set up the whole project yourself. No code is provided. Our test suite will run against your repo, so it must start with exactly these commands on a clean machine with Node.js 20+ and Docker:

```
docker compose up -d   # PostgreSQL 16
npm install
npm run db:reset       # create the schema and seed users
npm run dev            # API on :4000, React app on :5173
npm test               # your own tests
```

The React app must proxy `/api` to the API, so the browser only talks to port 5173. Run the server in UTC. The organisers and most applicants are in Singapore (UTC+8). Explain anything else we need in README.md.

Login stub. Real authentication is out of scope. Every API request except `/api/health` sends the header `X-User-Id` with a seeded user's ID. A missing or unknown ID returns 401. In the React app, opening any page with `?as=<userId>` signs in as that user.

Seed data. `npm run db:reset` must create exactly these users, with these IDs. Names are up to you. Create no programmes, entries or scores: our tests create their own.

Who	IDs	Organisation ID
Organisations	1 to 5	
Organisers	1, 2, 3	Same as the user ID
Judges	101 to 112	((id − 101) mod 5) + 1, so 101 is in org 1, 105 in org 5, 106 in org 1
Applicants	201 to 260	((id − 201) mod 5) + 1

API contract

Implement these endpoints exactly. Errors always return `{ "error": "<code>" }`, using the code shown where one is given. When a user asks for something they are not allowed to see, return 404 (403 is also accepted) and never return the data.

Method and path	Who may call it	Success	Errors
`GET /api/health`	Anyone, no header	200 `{ ok: true }`	
`GET /api/me`	Any user	200 `{ id, name, role, organisationId }`	401
`POST /api/programmes` `{ name }`	Organiser	201 `{ id, name, organiserId, judgingClosed }`	400, 403
`GET /api/programmes`	Organiser	200, their own programmes only	403
`POST /api/programmes/:id/categories` `{ name, deadlineLocal }`	Owning organiser	201 `{ id, programmeId, name, deadline }`	400, 404
`POST /api/programmes/:id/close-judging`	Owning organiser	200 programme	404
`GET /api/categories`	Any user	200, all categories	
`POST /api/entries` `{ categoryId, title, summary }`	Applicant	201 `{ id, categoryId, applicantId, title, summary, status: "draft", submittedAt: null }`	400, 403
`GET /api/entries/:id`	Its applicant, its assigned judges, the owning organiser	200 entry	404
`PATCH /api/entries/:id` `{ title?, summary? }`	Its applicant	200 entry	404; 409 `already_submitted`
`POST /api/entries/:id/submit`	Its applicant	200 entry with `status: "submitted"`	404; 409 `deadline_passed` or `already_submitted`
`POST /api/entries/:id/assignments` `{ judgeId }`	Owning organiser	201, or 200 if already assigned	400 if not a judge; 404; 409 `not_submitted`
`GET /api/judge/entries`	Judge	200 `[{ id, categoryId, title, myScore }]`, assigned entries only	403
`PUT /api/entries/:id/score` `{ score }`	Assigned judge	200 `{ entryId, judgeId, score }`	400; 404; 409 `judging_closed`
`GET /api/categories/:id/leaderboard?page=N`	Owning organiser	200 `{ page, pageSize: 20, totalEntries, totalPages, items: [{ entryId, title, averageScore, scoreCount }] }`	404
`GET /api/categories/:id/results.csv`	Owning organiser	200 CSV with header `entry_id,title,applicant_name,average_score,score_count`	404

Field formats:

- `deadlineLocal` is what the organiser typed, in Singapore time, as `YYYY-MM-DDTHH:mm`, for example `2026-10-31T23:59`.
- `deadline` and `submittedAt` are ISO 8601 UTC timestamps, for example `2026-10-31T15:59:00.000Z`.
- `averageScore` is rounded to 2 decimal places. The leaderboard lists submitted entries with at least one score, highest average first.

UI test hooks. Our browser test opens `http://localhost:5173/judge?as=<judgeId>` and uses these `data-testid` attributes: `judge-entry-<entryId>` on each entry in the judge's list, `entry-title` on the open entry's title, `score-input` and `score-save` on the scoring form, and `score-saved` on the message shown after a successful save.

Part A: Build mini-awards (45%)

Build a small award judging app to the spec below and the API contract above. Organisers run award programmes, applicants submit entries, and judges score them. Requirements are in priority order. Finish the Must items properly before starting the Should items.

ID	Priority	Requirement
R1	Must	An organiser creates a programme with categories. Each category has a submission deadline, entered as a Singapore date and time.
R2	Must	An applicant creates an entry (title, summary, category) and submits it before the deadline. "Deadline 31 October, 23:59" means 23:59 Singapore time. Submitted entries cannot be edited.
R3	Must	An organiser assigns judges to entries. A judge sees only the entries assigned to them.
R4	Must	A judge gives each assigned entry one score, a whole number from 1 to 10, and may change it until judging closes. Judges use venue Wi-Fi that drops often, and the client retries failed requests.
R5	Must	An organiser sees a leaderboard per category: entries ranked by average score, 20 per page. Ties are common.

ID	Priority	Requirement
R6	Must	An organiser sees and manages only their own programmes.
R7	Must	React screens for the judge: a list of assigned entries and a scoring form. Judges click quickly between entries on slow connections.
R8	Should	React screen for the organiser's leaderboard.
R9	Should	An organiser downloads results as a CSV file that they will open in Excel.

Non-functional requirements:

- Up to 20 judges score at the same time, and up to 400 entries per programme.
- A score must never be lost or counted twice.
- No endpoint may load an unbounded number of rows into memory.

Write tests for the parts you think are most likely to break. Record every assumption you made in DECISIONS.md.

Part B: Requirement change (15%)

On the morning of working day 3, we will email you a change request from the organiser. It is short and incomplete on purpose, just like real ones. Commit what you have, then handle it.

Before you write code, list in DECISIONS.md every question you would ask the organiser. Then pick an answer for each one, state it as an assumption, and build to it across the database, API and UI, with tests.

We grade the questions and assumptions as heavily as the code. We also look at how much of your Part A code you had to rewrite. A well-structured app absorbs the change in a few places.

Part C: Review AI-written code (15%)

The code below is from a pull request for a different version of mini-awards. It adds entry search and a CSV export for organisers. An AI agent wrote all of it. It passes lint and its own tests, and the product team wants it merged tomorrow.

In that codebase, `requireLogin` checks that `X-User-Id` belongs to a real user and sets `req.user`. `db.query` runs a query on a `pg` pool. The `users` table there also stores `email` and `phone`.

```
// routes/entries.js, added by feat/entry-search-export
router.get('/programmes/:programmeId/entries', requireLogin, async (req, res)
=> {
  const { q = '', sort = 'created_at', dir = 'desc' } = req.query;
  const { rows: entries } = await db.query(
    `SELECT e.id, e.title, e.status, u.name AS applicant_name, u.email,
u.phone
       FROM entries e JOIN users u ON u.id = e.applicant_id
      WHERE e.programme_id = $1 AND e.title ILIKE '%' || $2 || '%'
      ORDER BY ${sort} ${dir}`,
    [req.params.programmeId, q],
  );
  for (const entry of entries) {
    const { rows } = await db.query(
      'SELECT AVG(score)::float AS avg, COUNT(*)::int AS n FROM scores WHERE
entry_id = $1',
      [entry.id],
    );
    entry.averageScore = rows[0].avg;
    entry.scoreCount = rows[0].n;
  }
  res.json(entries);
});

router.get('/programmes/:programmeId/export.csv', requireLogin, async (req,
res) => {
  const { rows } = await db.query(
    `SELECT e.id, e.title, u.name, u.email, u.phone, AVG(s.score) AS avg
       FROM entries e
       JOIN users u ON u.id = e.applicant_id
       LEFT JOIN scores s ON s.entry_id = e.id
      WHERE e.programme_id = $1
      GROUP BY e.id, u.id`,
    [req.params.programmeId],
  );
  let csv = 'id,title,name,email,phone,average\n';
  for (const r of rows) {
    csv += [r.id, `"${r.title.replace(/"/g, '""')}"`, r.name, r.email,
```

```
r.phone, r.avg].join(',') + '\n';
  }
  res.header('Content-Type', 'text/csv').send(csv);
});
```

You are the last reviewer before it ships to production. Write REVIEW.md, 300 words at most:

1. List the problems you found, most serious first. For each one, give the file and line, what could go wrong in production, and who would be affected.
2. For the single most serious problem, write the corrected code in REVIEW.md.
3. End with a verdict: approve, approve with changes, or block. Give one sentence of reasoning.

Comments on naming, formatting or style score nothing. If you think the PR is fine, say so and defend it, but be sure.

This code may show you a mistake you also made in Part A. If it does, fix your own app and mention it in REVIEW.md. That earns credit.

Part D: Live defense (30 minutes, 25%)

You will share your screen and walk us through your submission. We will pick which parts to discuss. Expect to:

- explain any line we point at, and what breaks if we delete it
- run our concurrency script against your app and explain the result
- make a small live change we ask for, for about 10 minutes, with AI tools closed
- tell us one thing the AI got wrong and how you noticed

The defense is a gate. If you cannot explain a fix you submitted, that fix scores zero, however good the code is. "The AI wrote it" is an honest answer, but it is not an explanation.

Scoring and submission

A strong intern does not finish everything. They finish some of it correctly, prove it, and know where the AI misled them.

Part	Weight	What scores well	What scores poorly
A. Build	45%	Must items pass the hidden suite; rules enforced in the database and API; tests on the risky parts	Many screens that fail under concurrency; rules checked only in React; one giant commit
B. Requirement change	15%	Good questions; sensible stated assumptions; change lands in a few places	Silent assumptions; large rewrites of Part A
C. PR review	15%	Serious issues found and ranked by impact; top fix proven with a test	Style comments; a generic checklist that does not point at real lines
D. Live defense	25%	Explains own code; makes the live change; names a real AI mistake	Cannot explain submitted code (that work then scores zero)

Submit by the end of the 3.5th working day after you receive this test:

- `git bundle create <your-name>.bundle --all`
- README.md, DECISIONS.md, REVIEW.md and AI_LOG.md, committed at the repo root

Email the bundle to the hiring contact in your invitation. Late submissions are accepted but marked as late.